This example is trivial and only tests a very small simple function. The process is important, though, and it's always the same, as you've just seen.

Until now you've used a three-step process that included watching the test fail. You might consider this step as implied, making the process two steps:

1. Write a failing test for a single unit of code.

2. Write code that passes the test.

The important thing is that the test is written first. It's also very different from how you wrote the broken tracking application, which was solution first, test second. Actually, the tracking application never got to the second part, so there was no feedback about whether the application worked as it should. Test feedback matters.

10.2.4. Feedback

Currently, when a test passes you're blandly greeted with empty console output:

```
> coffee word_utils.spec.coffee
>
```

Empty console output

It would be useful to see feedback for each test stating whether it passed or failed. The following listing contains the tests for `addWord` and `removeWord` and a `fact` function that runs a test wrapped inside a `try...catch`.

Listing 10.1. Add and remove word